

UNIVERSIDAD DEL BOSQUE
Facultad de Ingeniería
Ingeniería de Software & Ingeniería en Robótica

Competitive Programming Notebook

Algoritmos, estructuras de datos y plantillas

C++ · Java · Python

Juan

Estudiante — 4.º semestre
Doble titulación: Software & Robótica

Bogotá, Colombia · 2026

Índice

I	Fundamentos	3
1	Entrada / Salida Rápida (Fast I/O)	3
1.1	C++ (g++ 8.1, -std=c++17)	3
1.2	Java	3
1.3	Python (3.7.4)	4
2	Formateo de Salida	4
2.1	C++ (printf)	4
2.2	Java (System.out.printf)	4
2.3	Python (3 formas)	4
3	Condicionales y Ciclos	4
3.1	C++	4
3.2	Java	4
3.3	Python (3.7.4)	4
4	Conversiones, Parseos y Casteos	5
4.1	C++	5
4.2	Java	5
4.3	Python (3.7.4)	5
II	Estructuras de Datos	5
5	Estructuras de Datos (Data Structures)	5
5.1	Arreglos (Arrays)	5
5.2	Matrices (Arreglos 2D)	5
5.3	Vectores / Listas dinámicas	5
5.4	Listas enlazadas (LinkedList)	5
5.5	Conjuntos hash (HashSet)	6
5.6	Mapas / Diccionarios hash	6
5.7	Conjuntos ordenados (TreeSet)	6
5.8	Mapas ordenados (TreeMap)	6
6	Disjoint Set Union (DSU)	6
III	Ordenamiento y Búsqueda	6
7	Ordenamiento y Búsqueda (Sorting & Searching)	6
8	Búsqueda Binaria (Binary Search)	6
IV	Matemáticas	6
9	Matemáticas (Math)	6
10	Teoría de Números (Number Theory)	6
11	Combinatoria (Combinatorics)	6
12	Máscaras de Bits (Bitmasking)	6
13	Geometría (Geometry)	6
V	Técnicas Algorítmicas	6
14	Programación Dinámica (Dynamic Programming)	6

15 Algoritmos Voraces (Greedy)	6
16 Ad-hoc y Simulación	7
 VI Grafos y Árboles	 7
17 Grafos (Graphs)	7
18 Árboles (Trees)	7
 VII Cadenas	 7
19 Cadenas (Strings)	7

Parte I Fundamentos

1 Entrada / Salida Rápida (Fast I/O)

De más lento (didáctico) a más rápido (para TLE ajustado). Usa el lector por bytes solo cuando haya del orden de 10^6 – 10^7 lecturas.

1.1 C++ (g++ 8.1, -std=c++17)

Tier 1 — Básico: cin / cout.

```
1 int entero; double decimal; string palabra, linea;
2 cin >> entero >> decimal >> palabra; // >> lee UN token
3 // ! cin >> deja el '\n' pendiente;
4 // ! el 1er getline sale VACÍO -> hay que ignorarlo
5 cin.ignore();
6 getline(cin, linea); // línea completa
```

Tier 2 — Rápido: desactiva la sync con C (una vez en main). Punto dulce para casi todo.

```
1 ios_base::sync_with_stdio(false);
2 cin.tie(nullptr); // ! no mezclar con scanf/printf
3 cout << entero << "\n"; // "\n", nunca endl (endl frena)
```

Tier 3 — Más rápido: lector por bytes con fread. Solo con 10^6 – 10^7 lecturas.

```
1 struct FastReader {
2     static const int TAM = 1 << 16; // 64 KB
3     char bufer[TAM];
4     int puntero = 0, leídos = 0;
5
6     int leer() {
7         if (puntero == leídos) {
8             leídos = (int) fread(bufer, 1, TAM, stdin);
9             puntero = 0;
10        }
11        return leídos == 0 ? -1 : bufer[puntero++]; // -1=EOF
12    }
13    int nextInt() {
14        int resultado = 0, b = leer();
15        while (b <= ' ') b = leer(); // saltar blancos
16        bool negativo = (b == '-');
17        if (negativo) b = leer();
18        while (b >= '0' && b <= '9') {
19            resultado = resultado * 10 + (b - '0'); b = leer();
20        }
21        return negativo ? -resultado : resultado;
22    }
23    long long nextLong() {
24        long long resultado = 0; int b = leer();
25        while (b <= ' ') b = leer();
26        bool negativo = (b == '-');
27        if (negativo) b = leer();
28        while (b >= '0' && b <= '9') {
29            resultado = resultado * 10 + (b - '0'); b = leer();
30        }
31        return negativo ? -resultado : resultado;
32    }
33    double nextDouble() {
34        double resultado = 0, divisor = 1; int b = leer();
35        while (b <= ' ') b = leer();
36        bool negativo = (b == '-');
37        if (negativo) b = leer();
38        while (b >= '0' && b <= '9') {
39            resultado = resultado * 10 + (b - '0'); b = leer();
40        }
41        if (b == '.') {
42            b = leer();
43            while (b >= '0' && b <= '9') {
44                resultado += (b - '0') / (divisor *= 10); b = leer();
45            }
46        }
47        return negativo ? -resultado : resultado;
48    }
49    string next() { // un token
50        string s; int b = leer();
51        while (b <= ' ') b = leer();
52        while (b > ' ') { s += (char) b; b = leer(); }
```

```
53        return s;
54    }
55    string nextLine() { // línea completa
56        string s; int b = leer();
57        while (b != '\n' && b != -1) {
58            if (b != '\r') s += (char) b; b = leer();
59        }
60        return s;
61    }
62 };
```

1.2 Java

Tier 1 — Básico: Scanner + System.out.

```
1 Scanner sc = new Scanner(System.in);
2 int entero = sc.nextInt();
3 String palabra = sc.next(); // un token
4 // ! tras nextInt/next, el 1er nextLine sale VACÍO:
5 sc.nextLine(); // quema la línea pendiente
6 String linea = sc.nextLine(); // ahora sí la línea real
```

Tier 2 — Intermedio: BufferedReader + StringTokenizer + PrintWriter. Estándar en CP.

```
1 BufferedReader lector = new BufferedReader(
2     new InputStreamReader(System.in));
3 PrintWriter escritor = new PrintWriter(
4     new BufferedWriter(new OutputStreamWriter(System.out)));
5
6 StringTokenizer separador =
7     new StringTokenizer(lector.readLine());
8 int primero = Integer.parseInt(separador.nextToken());
9 int segundo = Integer.parseInt(separador.nextToken());
10
11 escritor.println(primero + segundo);
12 escritor.flush(); // ! OBLIGATORIO al final, si no, no imprime
```

Tier 3 — StreamTokenizer: veloz para leer números.

```
1 StreamTokenizer in = new StreamTokenizer(
2     new BufferedReader(new InputStreamReader(System.in)));
3 in.nextToken();
4 int entero = (int) in.nval; // ! nval es double: ojo con > 2^53
```

Tier 4 — Más rápido: lector por bytes (DataInputStream). El más veloz sin librerías.

```
1 static class FastReader {
2     private final DataInputStream flujo;
3     private final byte[] bufer = new byte[1 << 16];
4     private int puntero = 0, leídos = 0;
5
6     FastReader() { flujo = new DataInputStream(System.in); }
7
8     int nextInt() throws IOException {
9         int resultado = 0, b = leer();
10        while (b <= ' ') b = leer();
11        boolean negativo = (b == '-');
12        if (negativo) b = leer();
13        while (b >= '0' && b <= '9') {
14            resultado = resultado * 10 + (b - '0'); b = leer();
15        }
16        return negativo ? -resultado : resultado;
17    }
18    long nextLong() throws IOException {
19        long resultado = 0; int b = leer();
20        while (b <= ' ') b = leer();
21        boolean negativo = (b == '-');
22        if (negativo) b = leer();
23        while (b >= '0' && b <= '9') {
24            resultado = resultado * 10 + (b - '0'); b = leer();
25        }
26        return negativo ? -resultado : resultado;
27    }
28    private int leer() throws IOException {
29        if (puntero == leídos) {
30            leídos = flujo.read(bufer, 0, bufer.length);
31            puntero = 0;
32        }
33        return leídos == -1 ? -1 : bufer[puntero++];
34    }
35 }
```

1.3 Python (3.7.4)

Tier 1 — Básico: `input()` / `print()`.

```
1 entero = int(input()) # lee UNA línea
2 primero, segundo = map(int, input().split())
```

Tier 2 — Intermedio: `sys.stdin/stdout` (más rápido que `input()`).

```
1 import sys
2 entrada = sys.stdin.readline
3 salida = sys.stdout.write
4 a, b = map(int, entrada().split())
5 salida(f"{a + b}\n")
```

Tier 3 — Más rápido: leer TODO el `stdin` de golpe y tokenizar; la salida, en UNA sola escritura.

```
1 import sys
2 def main():
3     datos = sys.stdin.buffer.read().split()
4     indice = 0
5     def siguiente_int(): # avanza por los tokens
6         nonlocal indice
7         valor = int(datos[indice]); indice += 1
8         return valor
9
10    n = siguiente_int()
11    total = 0
12    for _ in range(n):
13        total += siguiente_int()
14    sys.stdout.write(f"{total}\n")
15 main()
16
17 # Nota: en Python 32-bit ojo con la memoria (~2 GB).
18 # Para DFS recursivo: sys.setrecursionlimit(300000)
```

2 Formateo de Salida

2.1 C++ (printf)

```
1 printf("%d\n", 42); // 42 entero
2 printf("%lld\n", largo); // long long: SIEMPRE %lld
3 printf("%5d\n", 42); // " 42" ancho 5, derecha
4 printf("%-5d\n", 42); // "42 " izquierda
5 printf("%05d\n", 42); // 00042 rellena ceros
6 printf("%+d\n", 42); // +42 fuerza signo
7 printf("%x %X\n", 255, 255); // ff FF hexadecimal
8 printf("%.2f\n", 3.14159); // 3.14 (lo mas usado en CP)
9 printf("%.0f\n", 3.7); // 4 REDONDEA
10 printf("%e\n", 31415.9); // 3.141590e+04 cientifica
11 printf("%c %s\n", 'A', "hola");
12 printf("100%%\n"); // 100% %% = literal
13 // ! NO uses %n en C (peligroso). scanf: double con %lf
```

2.2 Java (System.out.printf)

```
1 System.out.printf("%d\n", 42); // 42
2 System.out.printf("%,d\n", 1000000); // 1,000,000 miles
3 System.out.printf("%05d\n", 42); // 00042
4 System.out.printf("%+d\n", 42); // +42
5 System.out.printf("%x\n", 255); // ff
6 System.out.printf("%.2f\n", 3.14159); // 3.14
7 System.out.printf("%-10s\n", "hi"); // "hi "
8 System.out.printf("%b\n", true); // true
9 // %n = salto portable (mejor que \n); %% = %
```

2.3 Python (3 formas)

```
1 # 1) operador % (estilo C)
2 print("%d %.2f" % (42, 3.14))
3 # 2) str.format
4 print("{:05d}".format(42)) # 00042
5 print("{:,}".format(1000000)) # 1,000,000
6 print("{:.1%}".format(0.25)) # 25.0%
7 # 3) f-strings (lo mas limpio, 3.6+)
8 x = 3.14159
9 print(f"{x:.2f}") # 3.14
10 print(f"{42:+d} {255:x}") # +42 ff
```

3 Condicionales y Ciclos

3.1 C++

```
1 if (x > 0) cout << "pos";
2 else if (x < 0) cout << "neg";
3 else cout << "cero";
4
5 string s = (x >= 0) ? "no neg" : "neg"; // ternario
6
7 switch (x) { // solo int/char, NO string
8     case 1: /*...*/ break; // ! sin break: fall-through
9     case 2: case 3: /*...*/ break; // varios case = OR
10    default: /*...*/ ;
11 }
12 if (int r = x % 2; r == 0) {} // C++17: if con init
13
14 for (int i = 0; i < n; i++) {}
15 for (int v : datos) {} // range-based (sin indice)
16 for (int &v : datos) v *= 2; // & modifica el original
17 while (c < n) c++;
18 do { c--; } while (c > 0); // ejecuta AL MENOS 1 vez
19 // break / continue.
20 // ! C++ NO tiene break con etiqueta: usa bandera o return
```

3.2 Java

```
1 if (x > 0) {} else if (x < 0) {} else {}
2
3 String s = (x >= 0) ? "no neg" : "neg"; // ternario
4
5 switch (x) {
6     case 1: /*...*/ break; // ! sin break: fall-through
7     case 2: case 3: /*...*/ break;
8     default: /*...*/ ;
9 }
10
11 for (int i = 0; i < n; i++) {}
12 for (int v : arreglo) {} // for-each
13 while (c < n) c++;
14 do { c--; } while (c > 0); // al menos 1 vez
15
16 externo: // EXCLUSIVO: break con etiqueta
17     for (int i = 0; i < 3; i++)
18         for (int j = 0; j < 3; j++)
19             if (i + j == 3) break externo; // rompe el externo
```

3.3 Python (3.7.4)

```
1 if x > 0:
2     pass
3 elif x < 0: # 'elif', no 'else if'
4     pass
5 else:
6     pass
7
8 s = "no neg" if x >= 0 else "neg" # ternario
9
10 # ! 3.7 NO tiene match/case (3.10). Usa dict de despacho:
11 opciones = {1: "uno", 2: "dos", 3: "tres"}
12 print(opciones.get(x, "otro")) # .get con default
13
14 for i in range(n): # range(inicio, fin, paso), fin excl.
15     pass
16 for v in datos: # for-each idiomático
17     pass
18 for i, v in enumerate(datos): # indice + valor
19     pass
20 while c < n:
21     c += 1
22 while True: # ! Python NO tiene do-while
23     c -= 1
24     if c <= 0:
25         break
26
27 for i in range(5): # EXCLUSIVO: for-else
28     if i == 99:
29         break
30 else: # corre solo si NO hubo break
31     print("sin break")
```

4 Conversiones, Parseos y Casteos

4.1 C++

```

1 // string -> numero (parseo)
2 int e = stoi("42");
3 long long l = stoll("10000000000");
4 double d = stod("3.14");
5 int bin = stoi("1010", nullptr, 2); // 10
6 int hex = stoi("ff", nullptr, 16); // 255
7 // numero -> string
8 string s = to_string(42); // "42"
9 // casteo: (int) TRUNCA hacia 0
10 int t = (int) 3.99; // 3
11 int r = static_cast<int>(3.99); // 3
12 // ! int/int = division ENTERA
13 double mal = 7 / 2; // 3.0
14 double bien = 7 / 2.0; // 3.5
15 // caracteres
16 int cod = (int) 'A'; // 65
17 char ch = (char) 66; // 'B'
18 int dig = '7' - '0'; // 7

```

4.2 Java

```

1 // String -> numero (parseo)
2 int e = Integer.parseInt("42");
3 long l = Long.parseLong("10000000000");
4 double d = Double.parseDouble("3.14");
5 int bin = Integer.parseInt("1010", 2); // 10
6 int hex = Integer.parseInt("ff", 16); // 255
7 // numero -> String
8 String s = String.valueOf(42); // "42"
9 String h = Integer.toHexString(255); // "ff"
10 // casteo
11 int t = (int) 3.99; // 3 (TRUNCA)
12 int r = (int) Math.round(3.99); // 4
13 double mal = 7 / 2; // 3.0
14 double bien = 7 / 2.0; // 3.5
15 // caracteres
16 int cod = (int) 'A'; // 65
17 char ch = (char) 66; // 'B'
18 int dig = '7' - '0'; // 7

```

4.3 Python (3.7.4)

```

1 # string -> numero
2 e = int("42"); d = float("3.14")
3 bin = int("1010", 2) # 10
4 hex = int("ff", 16) # 255
5 e2 = int(float("3.99")) # 3 (int("3.14") da ERROR)
6 # numero -> string
7 s = str(42) # "42"
8 b = format(10, "b") # "1010" (sin prefijo)
9 h = format(255, "x") # "ff"
10 # casteo
11 t = int(3.99) # 3 (TRUNCA hacia 0)
12 r = round(3.99) # 4
13 piso = 7 // 2 # 3
14 real = 7 / 2 # 3.5 (siempre float)
15 # caracteres
16 cod = ord("A") # 65
17 ch = chr(66) # "B"
18 dig = ord("7") - ord("0") # 7

```

Parte II Estructuras de Datos

5 Estructuras de Datos (Data Structures)

5.1 Arreglos (Arrays)

Colección de tamaño **fijo** del mismo tipo. **Complejidad:** acceso/asignación por índice $O(1)$; crear $O(n)$.

```

1 int a[5] = {1,2,3,4,5}; // crear: O(n)
2 a[0] = 10; // acceso/asignación: O(1)

```

```

1 int[] a = new int[5]; // crear: O(n)
2 a[0] = 10; // acceso: O(1)
3 int n = a.length; // O(1) (.length sin ())

```

```

1 a = [0] * 5 # crear: O(n) (no hay arreglo fijo)
2 a[0] = 10 # acceso: O(1)

```

5.2 Matrices (Arreglos 2D)

Arreglo bidimensional (filas $\times$ columnas). **Complejidad:** acceso $[i][j]$ $O(1)$; crear $n \times m$ en $O(nm)$.

```

1 vector<vector<int>> m(3, vector<int>(4,0)); // crear: O(n*m)
2 m[1][2] = 7; // acceso: O(1)

```

```

1 int[][] m = new int[3][4]; // crear: O(n*m)
2 m[1][2] = 7; // acceso: O(1)

```

```

1 m = [[0]*4 for _ in range(3)] # crear: O(n*m)
2 m[1][2] = 7 # acceso: O(1)
3 # ! NO [[0]*4]*3 (las filas se comparten)

```

5.3 Vectores / Listas dinámicas

Arreglo que **crece** solo (ArrayList / vector / list). **Complejidad:** acceso por índice $O(1)$; agregar al final $O(1)$ amortizado; insertar/borrar en medio $O(n)$; buscar $O(n)$.

```

1 vector<int> v;
2 v.push_back(3); // final: O(1) amortizado
3 int x = v[0]; // acceso: O(1)
4 int n = v.size(); // O(1)

```

```

1 ArrayList<Integer> v = new ArrayList<>();
2 v.add(3); // final: O(1) amortizado
3 int x = v.get(0); // acceso: O(1)
4 int n = v.size(); // O(1)

```

```

1 v = []
2 v.append(3) # final: O(1) amortizado
3 x = v[0] # acceso: O(1)
4 n = len(v) # O(1)

```

5.4 Listas enlazadas (LinkedList)

Nodos enlazados. **Complejidad:** insertar/eliminar en los **extremos** $O(1)$; acceso por índice $O(n)$; buscar $O(n)$. Útil como cola o deque.

```

1 list<int> l; // o deque<int>
2 l.push_front(1); // O(1)
3 l.push_back(9); // O(1) acceso [i]: O(n)

```

```

1 LinkedList<Integer> l = new LinkedList<>();
2 l.addFirst(1); // O(1)
3 l.addLast(9); // O(1)
4 l.removeFirst(); // O(1) get(i): O(n)

```

```

1 from collections import deque
2 d = deque()
3 d.appendleft(1) # O(1)
4 d.append(9) # O(1)
5 d.popleft() # O(1) d[i]: O(n)

```

5.5 Conjuntos hash (HashSet)

Colección **sin duplicados** y sin orden. **Complejidad:** insertar, buscar y borrar $O(1)$ promedio ($O(n)$ peor caso).

```
1 unordered_set<int> s;
2 s.insert(3); // O(1) prom.
3 bool hay = s.count(3); // O(1) prom.
```

```
1 HashSet<Integer> s = new HashSet<>();
2 s.add(3); // O(1) prom.
3 boolean hay = s.contains(3); // O(1) prom.
```

```
1 s = set()
2 s.add(3) # O(1) prom.
3 hay = 3 in s # O(1) prom.
```

5.6 Mapas / Diccionarios hash

Pares **clave** → **valor** sin orden (HashMap / unordered_map / dict). **Complejidad:** insertar, acceder y borrar por clave $O(1)$ promedio ($O(n)$ peor caso).

```
1 unordered_map<string,int> m;
2 m["a"] = 1; // insertar/asignar: O(1) prom.
3 int v = m["a"]; // acceso: O(1) prom.
```

```
1 HashMap<String,Integer> m = new HashMap<>();
2 m.put("a", 1); // O(1) prom.
3 int v = m.getOrDefault("a", 0); // O(1) prom.
```

```
1 m = {}
2 m["a"] = 1 # O(1) prom.
3 v = m.get("a", 0) # O(1) prom. (get con default)
```

5.7 Conjuntos ordenados (TreeSet)

Conjunto sin duplicados **mantenido ordenado** (árbol balanceado). **Complejidad:** insertar, buscar, borrar y piso/techo $O(\log n)$.

```
1 set<int> s;
2 s.insert(5); s.insert(1); // O(log n) c/u
3 int menor = *s.begin(); // min: O(1)
4 auto it = s.lower_bound(3); // techo: O(log n)
```

```
1 TreeSet<Integer> s = new TreeSet<>();
2 s.add(5); s.add(1); // O(log n) c/u
3 int menor = s.first(); // O(log n)
4 Integer techo = s.ceiling(3); // O(log n)
```

```
1 # ! Python NO trae TreeSet built-in
2 from sortedcontainers import SortedSet
3 s = SortedSet()
4 s.add(5); s.add(1) # O(log n) c/u
5 menor = s[0] # min: O(log n)
6 i = s.bisect_left(3) # techo: O(log n)
```

5.8 Mapas ordenados (TreeMap)

Mapa clave → valor **ordenado por clave** (árbol balanceado). **Complejidad:** insertar, acceder, borrar y floor/ceiling $O(\log n)$.

```
1 map<string,int> m;
2 m["b"] = 2; m["a"] = 1; // O(log n) c/u
3 auto primera = m.begin(); // clave menor: O(1)
```

```
1 TreeMap<String,Integer> m = new TreeMap<>();
2 m.put("b", 2); m.put("a", 1); // O(log n) c/u
3 String prim = m.firstKey(); // O(log n)
```

```
1 # ! dict NO ordena por clave (conserva insercion)
2 from sortedcontainers import SortedDict
3 m = SortedDict()
4 m["b"] = 2; m["a"] = 1 # O(log n) c/u
5 prim = m.peekitem(0) # menor clave: O(log n)
```

6 Disjoint Set Union (DSU)

Contenido en desarrollo.

Parte III Ordenamiento y Búsqueda

7 Ordenamiento y Búsqueda (Sorting & Searching)

Contenido en desarrollo.

8 Búsqueda Binaria (Binary Search)

Contenido en desarrollo.

Parte IV Matemáticas

9 Matemáticas (Math)

Contenido en desarrollo.

10 Teoría de Números (Number Theory)

Contenido en desarrollo.

11 Combinatoria (Combinatorics)

Contenido en desarrollo.

12 Máscaras de Bits (Bitmasking)

Contenido en desarrollo.

13 Geometría (Geometry)

Contenido en desarrollo.

Parte V Técnicas Algorítmicas

14 Programación Dinámica (Dynamic Programming)

Contenido en desarrollo.

15 Algoritmos Voraces (Greedy)

Contenido en desarrollo.

16 Ad-hoc y Simulación

Contenido en desarrollo.

Parte VI Grafos y Árboles

17 Grafos (Graphs)

Contenido en desarrollo.

18 Árboles (Trees)

Contenido en desarrollo.

Parte VII Cadenas

19 Cadenas (Strings)

Contenido en desarrollo.